Week3 →

Episodic SARSA with functn approx →

Recall function approximation

$$v_\pi(s) \approx \hat{v}(s,w) \doteq w^T x(s)$$

Similarly we can have, $q_\pi(s,a) \approx \hat{q}(s,a,w) \doteq w^T x(s,a)$

Episodic Semi-gradient SARSA for estimating $\hat{q} \approx q_*$

1) Input: a differentiable action-value function parameterization
$\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
2) Algorithm parameters: Step size $\alpha > 0$, small $\epsilon > 0$
3) Initialize value function weights $w \in \mathbb{R}^d$ (e.g $w = 0$)
4) Loop for each episode:
5) $S, A \leftarrow$ initial state and action of episode (e.g ϵ-greedy)
6) Loop for each step of episode:
7) Take action A, observe R and S'
If S' is terminal:
8)
9) $w \leftarrow w + \alpha[R - \hat{q}(S,A,w)] \cdot \nabla \hat{q}(S,A,w)$
Go to next episode
10)
11) Choose action A' as a function of $\hat{q}(S', \cdot, w)$

(e.g ε-greedy)

12) $w \leftarrow w + \alpha [R + \gamma \hat{q}(S', A, w) - \hat{q}(S, A, w)]. \nabla \hat{q}(S, A, w)$

13) $S \leftarrow S'$

14) $A \leftarrow A'$

Expected SARSA with function approximation →

SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$$

Expected SARSA:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \underbrace{\sum_{a'} \pi(a' | S_{t+1}) Q(S_{t+1}, a')}_{} - Q(S_t, A_t)\right)$$

Same expectation can be computed even when we use function approximation!

for expected SARSA :-

$$w \leftarrow w + \alpha \left(R_{t+1} + \gamma \sum_{a'} \pi(a' | S_{t+1}) \hat{q}(S_{t+1}, a', w) - \hat{q}(S_t, A_t, w)\right) \nabla \hat{q}(S_t, A_t, w)$$

for q-learning →

$$w \leftarrow w + \alpha\left(R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', w) - \hat{q}(S_t, A_t, w)\right). \nabla \hat{q}(S_t, A_t, w)$$

Exploration under function approximation →

Note: In optimistic initial values, we initialize our values to be greater than real values)

One of exploration technique like ϵ-greedy

In tabular setting, we initialize the value function this way, causes the agent to systematically explore the state action space.

In function approximation, optimistic initial values corresponds to initializing the weights such that the resulting values are optimistic.

Average Reward →

\# Imagine that agent has interacted with the world for H steps. This is the reward it has received on average across those H-steps. In other words, its rate of reward.

$$\boxed{r(\pi) \doteq \lim_{h\to\infty} \sum_{t=1}^{h} E\left[R_t \mid S_0, A_{0:t-1} \sim \pi\right]}$$

\# If the agents goal is to maximise this average reward, then it cares equally about nearby and distant rewards.

\# More generally, we can write the avesge ~~reward~~ reward using the state visitation "μ".

$$r(\pi) = \sum_s \mu_\pi(s) \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) \times r$$

\# The average reward defination is intutive for saying if one policy is better than another, but how can we decide which actions from a state are better.?

In the average reward setting, returns are defined in terms of difference b/w rewards and the average reward $r(\pi)$.

$$G_t = [R_{t+1} - r(\pi)] + [R_{t+2} - r(\pi)] + \dots$$ } Called the differential return!

The differential return represents how much more reward the agent will receive from the current state in action compared to the average reward of the policy.

differential return represents how much better it is to take an action in a state then on average under the certain policy!

The differential return can only be used to compare actions if the same policy is followed on subsequent steps.

$$q_\pi(s,a) = E_\pi[G_t \mid S_t = s, A_t = a]$$

⇒ This quantity captures how much more reward the agent will get by starting in a particular state than it would get on average over all states if it followed a fixed policy!

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a) \left[r - r(\pi) + \sum_{a} \pi(a' \mid s')\, q_\pi(s',a') \right]$$

Differential semi-gradient SARSA for estimating $\hat{q} \approx q_*$

---

1) Input: a differentiable action value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \rightarrow \mathbb{R}$

2) Algorithm parameters: step sizes $\alpha > 0$, $\beta > 0$

3) Initialize value function weights $w \in \mathbb{R}^d$ arbitrarily e.g ($w = 0$)

3.1) Initialize average reward estimate $\bar{R} \in \mathbb{R}$ arbitrarily

4) Initialize state S, and Action A

5) Loop for each step:

6) Take action A, observe R, S'

7) Choose A' as function of $\hat{q}(S', \cdot, w)$ (e.g ϵ-greedy)

8) $\delta \leftarrow R - \bar{R} + \hat{q}(S', A', w) - \hat{q}(S, A, w)$

9) $\bar{R} \leftarrow \bar{R} + \beta(R - \bar{R})$

10) $w \leftarrow w + \alpha \delta \nabla q(S, A, w)$

11) $S \leftarrow S'$

12) $A \leftarrow A'$